

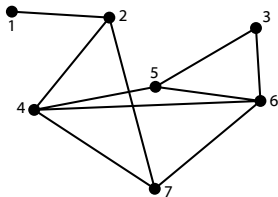
Grafos

Algoritmos y Estructuras de Datos III

Departamento de Computación
FCEyN, Universidad de Buenos Aires

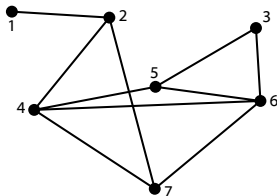
Definiciones básicas

- Un grafo G está formado por un par $(V(G), E(G))$:



Definiciones básicas

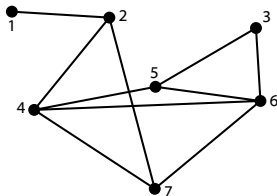
- Un grafo G está formado por un par $(V(G), E(G))$:
 - $V(G)$ es un conjunto finito, el conjunto de vértices de G , y



$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

Definiciones básicas

- Un **grafo** G está formado por un par $(V(G), E(G))$:
 - $V(G)$ es un conjunto finito, el conjunto de **vértices** de G , y
 - $E(G)$ es un conjunto de pares no ordenados de vértices distintos de G , llamados **aristas**, que se notan por ij o (i, j) .

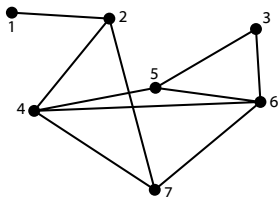


$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G) = \{(1, 2), (2, 4), (2, 7), (3, 5), (3, 6), (4, 5), (4, 6), (4, 7), (5, 6), (6, 7)\}$$

Definiciones básicas

- Un **grafo** G está formado por un par $(V(G), E(G))$:
 - $V(G)$ es un conjunto finito, el conjunto de **vértices** de G , y
 - $E(G)$ es un conjunto de pares no ordenados de vértices distintos de G , llamados **aristas**, que se notan por ij o (i, j) .
- **Notación:**

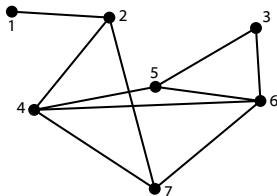


$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G) = \{(1, 2), (2, 4), (2, 7), (3, 5), (3, 6), (4, 5), (4, 6), (4, 7), (5, 6), (6, 7)\}$$

Definiciones básicas

- Un **grafo** G está formado por un par $(V(G), E(G))$:
 - $V(G)$ es un conjunto finito, el conjunto de **vértices** de G , y
 - $E(G)$ es un conjunto de pares no ordenados de vértices distintos de G , llamados **aristas**, que se notan por ij o (i, j) .
- **Notación:**
 - $n = n_G = |V(G)|$ y $m = m_G = |E(G)|$;



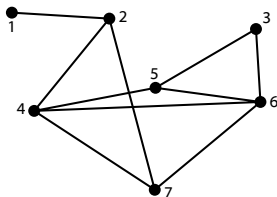
$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G) = \{(1, 2), (2, 4), (2, 7), (3, 5), (3, 6), (4, 5), (4, 6), (4, 7), (5, 6), (6, 7)\}$$

$$n = 7; m = 10.$$

Definiciones básicas

- Un **grafo** G está formado por un par $(V(G), E(G))$:
 - $V(G)$ es un conjunto finito, el conjunto de **vértices** de G , y
 - $E(G)$ es un conjunto de pares no ordenados de vértices distintos de G , llamados **aristas**, que se notan por ij o (i, j) .
- **Notación:**
 - $n = n_G = |V(G)|$ y $m = m_G = |E(G)|$;
 - $V_G = V(G)$, $E_G = E(G)$.



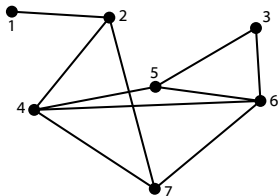
$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G) = \{(1, 2), (2, 4), (2, 7), (3, 5), (3, 6), (4, 5), (4, 6), (4, 7), (5, 6), (6, 7)\}$$

$$n = 7; m = 10.$$

Definiciones básicas

- Un **grafo** G está formado por un par $(V(G), E(G))$:
 - $V(G)$ es un conjunto finito, el conjunto de **vértices** de G , y
 - $E(G)$ es un conjunto de pares no ordenados de vértices distintos de G , llamados **aristas**, que se notan por ij o (i, j) .
- **Notación:**
 - $n = n_G = |V(G)|$ y $m = m_G = |E(G)|$;
 - $V_G = V(G)$, $E_G = E(G)$.
- Un grafo se dice **trivial** si tiene un solo vértice.



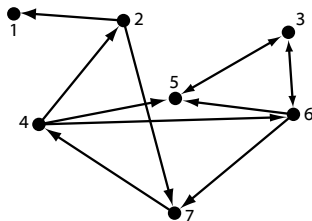
$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G) = \{(1, 2), (2, 4), (2, 7), (3, 5), (3, 6), (4, 5), (4, 6), (4, 7), (5, 6), (6, 7)\}$$

$$n = 7; m = 10.$$

Definiciones básicas

- Decimos que G es un **digrafo**, o un grafo dirigido, si las aristas están dadas por un conjunto de **pares ordenados** de vértices.



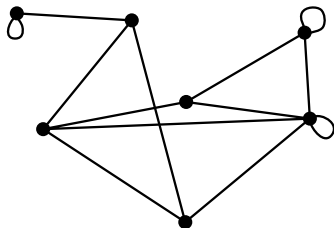
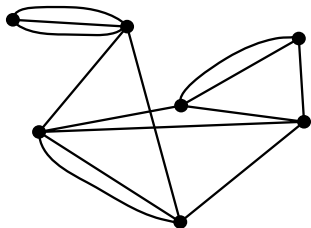
$$V(G) = \{1, 2, 3, 4, 5, 6, 7\}$$

$$E(G) = \{(2, 1), (4, 2), (2, 7), (3, 5), (5, 3), (3, 6), (6, 3), (4, 5), (4, 6), (7, 4), (6, 5), (6, 7)\}$$

$$n = 7; m = 12.$$

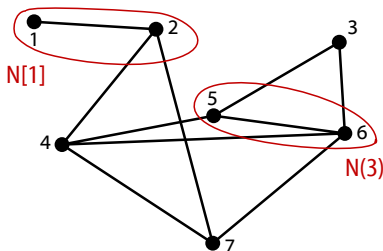
Definiciones básicas

- Decimos que G es un **multigrafo** si se permite que entre un mismo par de vértices se trace más de una arista, y un **pseudografo** si se permiten aristas de tipo (v, v) (**loops**).



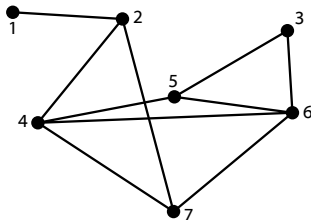
Vecindarios

- Un vértice v es **adyacente** a otro vértice w en G si $(v, w) \in E(G)$. Decimos que v y w son los **extremos** de la arista.
- El vecindario de un vértice v en un grafo G es el conjunto $N_G(v)$ que consiste de todos los vértices adyacentes a v . El vecindario cerrado de v es $N_G[v] = N_G(v) \cup \{v\}$.
- Notación:** si queda claro por contexto, se usa $N(v)$ y $N[v]$.



Grado

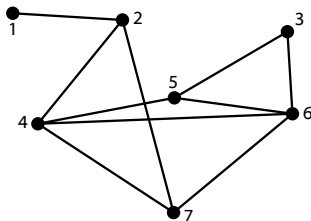
- El **grado** de un vértice v en G es la cardinalidad del conjunto $N_G(v)$ y se nota $d_G(v)$. Si no hay ambigüedad, se usa $d(v)$.



$$d(2) = 3$$

Grado

- El **grado** de un vértice v en G es la cardinalidad del conjunto $N_G(v)$ y se nota $d_G(v)$. Si no hay ambigüedad, se usa $d(v)$.
- Dado un grafo G , notamos $\delta(G)$ al grado mínimo y $\Delta(G)$ al grado máximo entre los vértices de G .



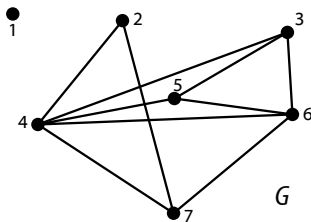
$$d(2) = 3$$

$$\delta(G) = 1$$

$$\Delta(G) = 4$$

Grado

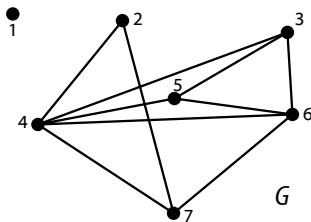
- Un vértice v es **aislado** cuando $N(v) = \emptyset$, o equivalentemente $d(v) = 0$.



El vértice 1 es aislado en G .

Grado

- Un vértice v es **aislado** cuando $N(v) = \emptyset$, o equivalentemente $d(v) = 0$.
- Un vértice v es **universal** cuando $N(v) = V(G) - \{v\}$, o equivalentemente $d(v) = n - 1$.

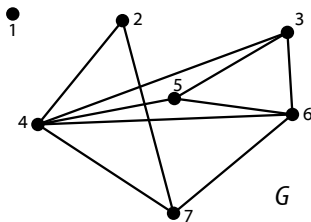


El vértice 1 es aislado en G .

El vértice 4 es universal en $G - \{1\}$.

Grado

- Un vértice v es **aislado** cuando $N(v) = \emptyset$, o equivalentemente $d(v) = 0$.
- Un vértice v es **universal** cuando $N(v) = V(G) - \{v\}$, o equivalentemente $d(v) = n - 1$.



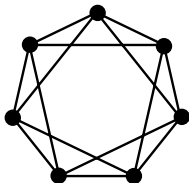
El vértice 1 es aislado en G .

El vértice 4 es universal en $G - \{1\}$.

Si G es no trivial y tiene un vértice aislado no puede tener también uno universal.

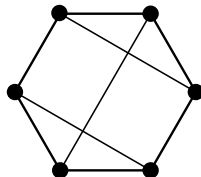
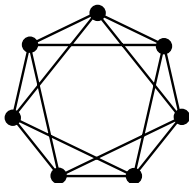
Grado

- Un grafo se dice **regular** si todos sus vértices tienen el mismo grado.



Grado

- Un grafo se dice **regular** si todos sus vértices tienen el mismo grado.
- Un grafo se dice **cúbico** si todos sus vértices tienen grado **tres**.



Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Demo: Por inducción en m_G .

Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Demo: Por inducción en m_G . Si $m_G = 0$, entonces $d_G(v) = 0$ para todo $v \in V(G)$, y por lo tanto $0 = \sum_{v \in V(G)} d(v) = 2m$.

Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Demo: Por inducción en m_G . Si $m_G = 0$, entonces $d_G(v) = 0$ para todo $v \in V(G)$, y por lo tanto $0 = \sum_{v \in V(G)} d(v) = 2m$. Supongamos $m_G > 0$, y consideremos G' obtenido a partir de G sacando una arista cualquiera (i, j) .

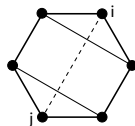
Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Demo: Por inducción en m_G . Si $m_G = 0$, entonces $d_G(v) = 0$ para todo $v \in V(G)$, y por lo tanto $0 = \sum_{v \in V(G)} d(v) = 2m$. Supongamos $m_G > 0$, y consideremos G' obtenido a partir de G sacando una arista cualquiera (i, j) . Entonces:

- $m_{G'} = m_G - 1$
- $d_{G'}(i) = d_G(i) - 1$ y $d_{G'}(j) = d_G(j) - 1$
- $d_{G'}(v) = d_G(v)$ si $v \neq i, j$



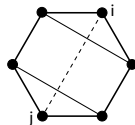
Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Demo: Por inducción en m_G . Si $m_G = 0$, entonces $d_G(v) = 0$ para todo $v \in V(G)$, y por lo tanto $0 = \sum_{v \in V(G)} d(v) = 2m$. Supongamos $m_G > 0$, y consideremos G' obtenido a partir de G sacando una arista cualquiera (i, j) . Entonces:

- $m_{G'} = m_G - 1$
- $d_{G'}(i) = d_G(i) - 1$ y $d_{G'}(j) = d_G(j) - 1$
- $d_{G'}(v) = d_G(v)$ si $v \neq i, j$



Por hipótesis inductiva, $\sum_{v \in V(G')} d_{G'}(v) = 2m_{G'}$. Luego
 $d_{G'}(i) + d_{G'}(j) + \sum_{v \in V(G'), v \neq i, j} d_{G'}(v) = 2m_{G'}$.

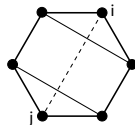
Grado

Teorema

$$\sum_{v \in V(G)} d(v) = 2m.$$

Demo: Por inducción en m_G . Si $m_G = 0$, entonces $d_G(v) = 0$ para todo $v \in V(G)$, y por lo tanto $0 = \sum_{v \in V(G)} d(v) = 2m$. Supongamos $m_G > 0$, y consideremos G' obtenido a partir de G sacando una arista cualquiera (i, j) . Entonces:

- $m_{G'} = m_G - 1$
- $d_{G'}(i) = d_G(i) - 1$ y $d_{G'}(j) = d_G(j) - 1$
- $d_{G'}(v) = d_G(v)$ si $v \neq i, j$



Por hipótesis inductiva, $\sum_{v \in V(G')} d_{G'}(v) = 2m_{G'}$. Luego $d_{G'}(i) + d_{G'}(j) + \sum_{v \in V(G'), v \neq i, j} d_{G'}(v) = 2m_{G'}$. Reemplazando, $d_G(i) - 1 + d_G(j) - 1 + \sum_{v \in V(G), v \neq i, j} d_G(v) = 2(m_G - 1)$. Es decir, $(\sum_{v \in V(G)} d(v)) - 2 = 2m_G - 2$ y por lo tanto $\sum_{v \in V(G)} d(v) = 2m_G$. $\square$

Grado

Corolario

Todo grafo cúbico tiene un número par de vértices.

Grado

Corolario

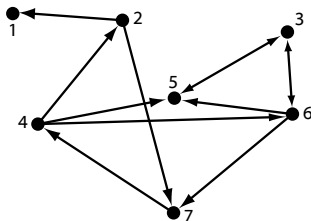
Todo grafo cúbico tiene un número par de vértices.

Demo: $2m = \sum_{v \in V(G)} d(v) = 3n$. Luego $2 \mid n$.



Grados en digrafos

- El **grado de entrada** $d_{in}(v)$ de un vértice v de un grafo orientado es la cantidad de arcos que **llegan** a v . Es decir, la cantidad de arcos que tienen a v como segundo elemento.
- El **grado de salida** $d_{out}(v)$ de un vértice v de un grafo orientado es la cantidad de arcos que **salen** de v . Es decir, la cantidad de arcos que tienen a v como primer elemento.



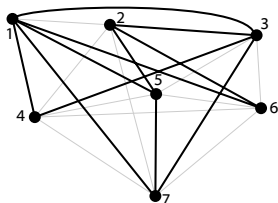
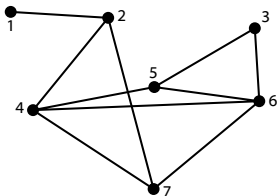
$$d_{in}(4) = 1$$

$$d_{out}(4) = 3$$

$$\sum_{v \in V(G)} d_{in}(v) = \sum_{v \in V(G)} d_{out}(v)$$

Complemento

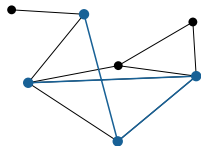
- El **complemento** de un grafo G , denotado por $\overline{G}$, es el grafo que tiene el mismo conjunto de vértices de G y tal que dos vértices distintos son adyacentes en $\overline{G}$ si y sólo si no son adyacentes en G .



Si G tiene n vértices y m aristas, ¿cuántas aristas tiene $\overline{G}$?

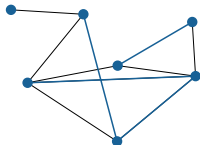
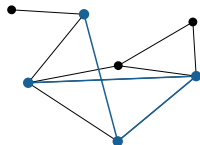
Subgrafos

- Un grafo H es un **subgrafo** de un grafo G si $V(H) \subseteq V(G)$ y $E(H) \subseteq E(G)$.



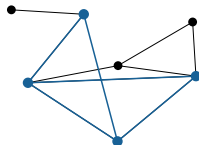
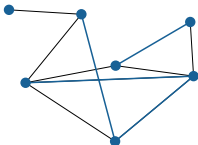
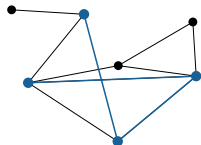
Subgrafos

- Un grafo H es un **subgrafo** de un grafo G si $V(H) \subseteq V(G)$ y $E(H) \subseteq E(G)$.
- Si $V(H) = V(G)$, decimos que H es un **subgrafo generador** de G .



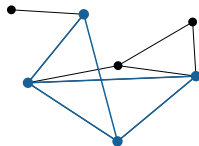
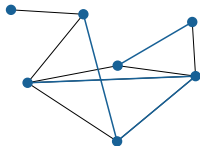
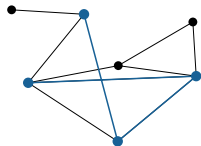
Subgrafos

- Un grafo H es un **subgrafo** de un grafo G si $V(H) \subseteq V(G)$ y $E(H) \subseteq E(G)$.
- Si $V(H) = V(G)$, decimos que H es un **subgrafo generador** de G .
- Dado un conjunto de vértices $X \subseteq V(G)$, el subgrafo de G **inducido** por X es el subgrafo H de G tal que $V(H) = X$ y $E(H)$ es el conjunto de aristas de G que tiene ambos extremos en X .



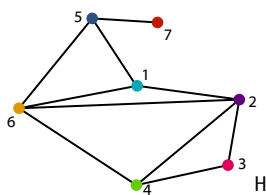
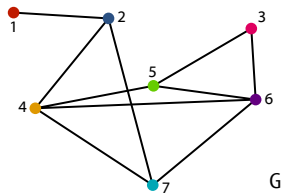
Subgrafos

- Un grafo H es un **subgrafo** de un grafo G si $V(H) \subseteq V(G)$ y $E(H) \subseteq E(G)$.
- Si $V(H) = V(G)$, decimos que H es un **subgrafo generador** de G .
- Dado un conjunto de vértices $X \subseteq V(G)$, el subgrafo de G **inducido** por X es el subgrafo H de G tal que $V(H) = X$ y $E(H)$ es el conjunto de aristas de G que tiene ambos extremos en X .
- Notación:** Si $v \in V(G)$, $G - v$ denota el subgrafo de G inducido por $V(G) - \{v\}$.



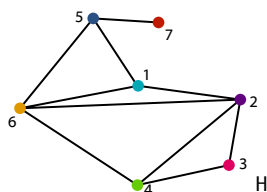
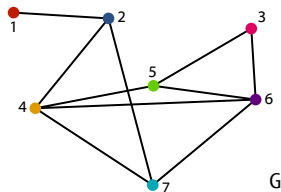
Isomorfismo

- Dos grafos G y H son isomorfos si existe una biyección entre $V(G)$ y $V(H)$ que conserva las adyacencias. En este caso, notamos $G = H$.



Isomorfismo

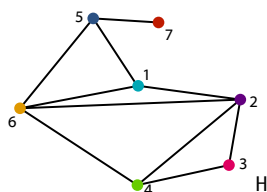
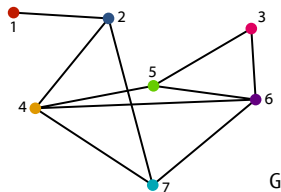
- Dos grafos G y H son isomorfos si existe una biyección entre $V(G)$ y $V(H)$ que conserva las adyacencias. En este caso, notamos $G = H$.
- Más formalmente, G y H son isomorfos si existe $f : V(G) \rightarrow V(H)$ biyectiva tal que $(v, w) \in E(G)$ si y sólo si $(f(v), f(w)) \in E(H)$.



$$\begin{aligned} f(1) &= 7 \\ f(2) &= 5 \\ f(3) &= 3 \\ f(4) &= 6 \\ f(5) &= 4 \\ f(6) &= 2 \\ f(7) &= 1 \end{aligned}$$

Isomorfismo

- Dos grafos G y H son isomorfos si existe una biyección entre $V(G)$ y $V(H)$ que conserva las adyacencias. En este caso, notamos $G = H$.
- Más formalmente, G y H son isomorfos si existe $f : V(G) \rightarrow V(H)$ biyectiva tal que $(v, w) \in E(G)$ si y sólo si $(f(v), f(w)) \in E(H)$.
- El isomorfismo es una relación de equivalencia.



$$\begin{aligned}
 f(1) &= 7 \\
 f(2) &= 5 \\
 f(3) &= 3 \\
 f(4) &= 6 \\
 f(5) &= 4 \\
 f(6) &= 2 \\
 f(7) &= 1
 \end{aligned}$$

Isomorfismo

Si dos grafos $G = (V, X)$ y $G' = (V', X')$ son isomorfos, entonces

- tienen el mismo número de vértices,
- tienen el mismo número de aristas,
- para todo k , $0 \leq k \leq n - 1$, tienen el mismo número de vértices de grado k ,
- tienen el mismo número de componentes conexas,
- para todo k , $1 \leq k \leq n - 1$, tienen el mismo número de caminos simples de longitud k .

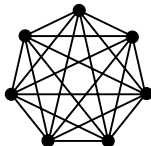
Isomorfismo

¿Es cierta la recíproca de esta propiedad?

¿Hay condiciones necesarias y suficientes fácilmente verificables para ver si dos grafos son isomorfos?

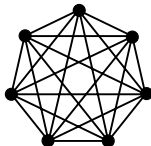
Grafos completos

- Un grafo G es **completo** si cualquier par de vértices distintos de G son adyacentes. Llamamos K_n al grafo completo con n vértices.



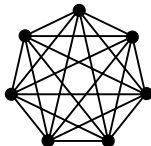
Grafos completos

- Un grafo G es **completo** si cualquier par de vértices distintos de G son adyacentes. Llamamos K_n al grafo completo con n vértices.
- K_3 se llama también **triángulo**.



Grafos completos

- Un grafo G es **completo** si cualquier par de vértices distintos de G son adyacentes. Llamamos K_n al grafo completo con n vértices.
- K_3 se llama también **triángulo**.
- ¿Cuánto valen m_{K_n} , $\delta(K_n)$ y $\Delta(K_n)$?



Caminos

- Un **camino** en un grafo G es una secuencia de vértices distintos $P = v_1, v_2, \dots, v_k$, donde $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k - 1$.



Caminos

- Un **camino** en un grafo G es una secuencia de vértices distintos $P = v_1, v_2, \dots, v_k$, donde $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k - 1$.
- Una **cuerda** en P es una arista que une dos vértices no consecutivos de P .



Caminos

- Un **camino** en un grafo G es una secuencia de vértices distintos $P = v_1, v_2, \dots, v_k$, donde $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k - 1$.
- Una **cuerda** en P es una arista que une dos vértices no consecutivos de P .
- Un **camino inducido** es un camino sin cuerdas. Denotamos por P_k al camino inducido de k vértices.



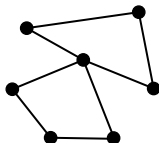
Caminos

- Un **camino** en un grafo G es una secuencia de vértices distintos $P = v_1, v_2, \dots, v_k$, donde $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k - 1$.
- Una **cuerda** en P es una arista que une dos vértices no consecutivos de P .
- Un **camino inducido** es un camino sin cuerdas. Denotamos por P_k al camino inducido de k vértices.
- ¿Cuánto valen m_{P_k} , $\delta(P_k)$ y $\Delta(P_k)$?



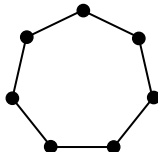
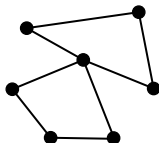
Circuitos y ciclos

- Un **circuito** en un grafo G es una secuencia de vértices $C = v_1, v_2, \dots, v_k, v_{k+1}$, no necesariamente distintos, donde $v_1 = v_{k+1}$ y $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k$.



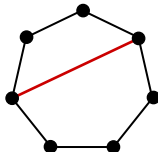
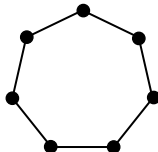
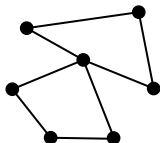
Circuitos y ciclos

- Un **circuito** en un grafo G es una secuencia de vértices $C = v_1, v_2, \dots, v_k, v_{k+1}$, no necesariamente distintos, donde $v_1 = v_{k+1}$ y $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k$.
- Si $k \geq 3$ y $v_1, \dots, v_k$ son distintos, C se llama **ciclo**.



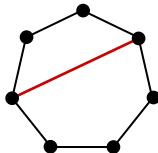
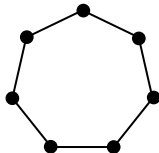
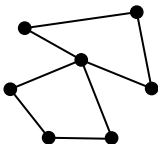
Circuitos y ciclos

- Un **circuito** en un grafo G es una secuencia de vértices $C = v_1, v_2, \dots, v_k, v_{k+1}$, no necesariamente distintos, donde $v_1 = v_{k+1}$ y $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k$.
- Si $k \geq 3$ y $v_1, \dots, v_k$ son distintos, C se llama **ciclo**.
- Una **cuerda** en C es cualquier cuerda del camino $v_1, v_2, \dots, v_k$ excepto (v_1, v_k) .



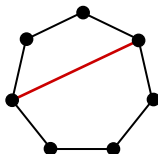
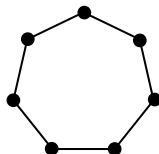
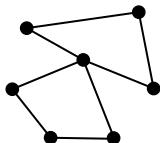
Circuitos y ciclos

- Un **circuito** en un grafo G es una secuencia de vértices $C = v_1, v_2, \dots, v_k, v_{k+1}$, no necesariamente distintos, donde $v_1 = v_{k+1}$ y $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k$.
- Si $k \geq 3$ y $v_1, \dots, v_k$ son distintos, C se llama **ciclo**.
- Una **cuerda** en C es cualquier cuerda del camino $v_1, v_2, \dots, v_k$ excepto (v_1, v_k) .
- Un ciclo es un **ciclo inducido** si no posee cuerdas. Llamamos C_k al ciclo inducido de k vértices.



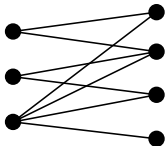
Circuitos y ciclos

- Un **circuito** en un grafo G es una secuencia de vértices $C = v_1, v_2, \dots, v_k, v_{k+1}$, no necesariamente distintos, donde $v_1 = v_{k+1}$ y $(v_i, v_{i+1}) \in E(G)$, $i = 1, \dots, k$.
- Si $k \geq 3$ y $v_1, \dots, v_k$ son distintos, C se llama **ciclo**.
- Una **cuerda** en C es cualquier cuerda del camino $v_1, v_2, \dots, v_k$ excepto (v_1, v_k) .
- Un ciclo es un **ciclo inducido** si no posee cuerdas. Llamamos C_k al ciclo inducido de k vértices.
- ¿Cuánto valen m_{C_k} , $\delta(C_k)$ y $\Delta(C_k)$?



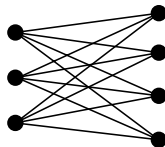
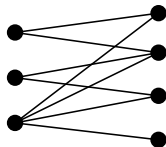
Grafos bipartitos completos

- Un grafo G es **bipartito** si $V(G) = V_1 \cup V_2$, con V_1 y V_2 disjuntos, y toda arista tiene un extremo en V_1 y otro en V_2 .



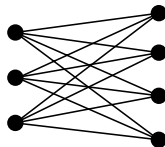
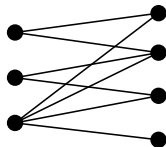
Grafos bipartitos completos

- Un grafo G es **bipartito** si $V(G) = V_1 \cup V_2$, con V_1 y V_2 disjuntos, y toda arista tiene un extremo en V_1 y otro en V_2 .
- Un grafo G es **bipartito completo** si además todo vértice de V_1 es adyacente a todo vértice de V_2 . Llamamos $K_{r,s}$ al grafo bipartito completo tal que $|V_1| = r$ y $|V_2| = s$.



Grafos bipartitos completos

- Un grafo G es **bipartito** si $V(G) = V_1 \cup V_2$, con V_1 y V_2 disjuntos, y toda arista tiene un extremo en V_1 y otro en V_2 .
- Un grafo G es **bipartito completo** si además todo vértice de V_1 es adyacente a todo vértice de V_2 . Llamamos $K_{r,s}$ al grafo bipartito completo tal que $|V_1| = r$ y $|V_2| = s$.
- ¿Cuánto valen $n_{K_{r,s}}$, $m_{K_{r,s}}$, $\delta(K_{r,s})$ y $\Delta(K_{r,s})$?



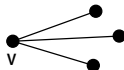
Teorema

Si un grafo tiene 6 o más vértices, entonces el grafo o su complemento tienen un triángulo.

Teorema

Si un grafo tiene 6 o más vértices, entonces el grafo o su complemento tienen un triángulo.

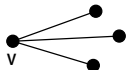
Demo: Sea $v \in V(G)$. Como $d_G(v) + d_{\bar{G}}(v) = n - 1 \geq 5$, podemos asumir s.p.g. que $d_G(v) \geq 3$.



Teorema

Si un grafo tiene 6 o más vértices, entonces el grafo o su complemento tienen un triángulo.

Demo: Sea $v \in V(G)$. Como $d_G(v) + d_{\bar{G}}(v) = n - 1 \geq 5$, podemos asumir s.p.g. que $d_G(v) \geq 3$.

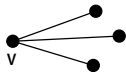


Si hay dos vértices adyacentes w y z en $N_G(v)$, entonces v, w, z forman un triángulo.

Teorema

Si un grafo tiene 6 o más vértices, entonces el grafo o su complemento tienen un triángulo.

Demo: Sea $v \in V(G)$. Como $d_G(v) + d_{\bar{G}}(v) = n - 1 \geq 5$, podemos asumir s.p.g. que $d_G(v) \geq 3$.



Si hay dos vértices adyacentes w y z en $N_G(v)$, entonces v, w, z forman un triángulo. Si no hay dos vértices adyacentes en $N_G(v)$, entonces $N_G(v)$ induce un subgrafo completo en $\bar{G}$, y como $|N_G(v)| \geq 3$, $\bar{G}$ contiene un triángulo.

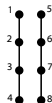
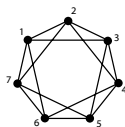
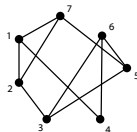
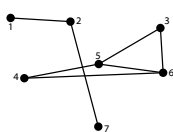


Conexión

- Un grafo G es **conexo** si para todo par de vértices distintos v y w de G existe un camino de v a w .

Conexión

- Un grafo G es **conexo** si para todo par de vértices distintos v y w de G existe un camino de v a w .
- ¿Cuáles de los siguientes grafos son conexos?



Conexión

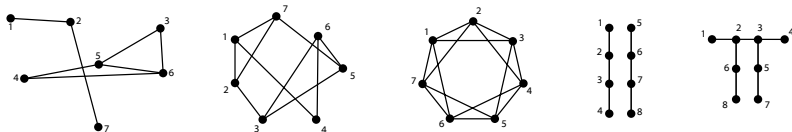
- Un conjunto S es **maximal** (**minimal**) en relación a una determinada propiedad P si S satisface P , y todo conjunto S' que contiene propiamente a S (que está contenido propiamente en S) no satisface P .

Conexión

- Un conjunto S es **maximal** (**minimal**) en relación a una determinada propiedad P si S satisface P , y todo conjunto S' que contiene propiamente a S (que está contenido propiamente en S) no satisface P .
- Una **componente conexa** de un grafo es un subgrafo conexo maximal.

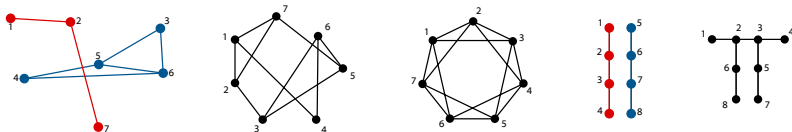
Conexión

- Un conjunto S es **maximal** (**minimal**) en relación a una determinada propiedad P si S satisface P , y todo conjunto S' que contiene propiamente a S (que está contenido propiamente en S) no satisface P .
- Una **componente conexa** de un grafo es un subgrafo conexo maximal.
- ¿Cuáles son las componentes conexas de estos grafos?



Conexión

- Un conjunto S es **maximal** (**minimal**) en relación a una determinada propiedad P si S satisface P , y todo conjunto S' que contiene propiamente a S (que está contenido propiamente en S) no satisface P .
- Una **componente conexa** de un grafo es un subgrafo conexo maximal.
- ¿Cuáles son las componentes conexas de estos grafos?



Conexión

Observaciones

1. Todo vértice de un grafo pertenece a alguna componente conexa.
2. Un grafo es conexo si y sólo si tiene una sola componente conexa.
3. Dos componentes conexas distintas de un grafo son disjuntas.

Conexión

Observaciones

1. Todo vértice de un grafo pertenece a alguna componente conexa.
2. Un grafo es conexo si y sólo si tiene una sola componente conexa.
3. Dos componentes conexas distintas de un grafo son disjuntas.

Demo de 3.: Supongamos que $v \in G_1 \cap G_2$.

Conexión

Observaciones

1. Todo vértice de un grafo pertenece a alguna componente conexa.
2. Un grafo es conexo si y sólo si tiene una sola componente conexa.
3. Dos componentes conexas distintas de un grafo son disjuntas.

Demo de 3.: Supongamos que $v \in G_1 \cap G_2$. Entonces para todo par de vértices w, z de $G_1 \cup G_2$ existe un camino de w a v y un camino de v a z (de longitud cero si alguno es v).



Conexión

Observaciones

1. Todo vértice de un grafo pertenece a alguna componente conexa.
2. Un grafo es conexo si y sólo si tiene una sola componente conexa.
3. Dos componentes conexas distintas de un grafo son disjuntas.

Demo de 3.: Supongamos que $v \in G_1 \cap G_2$. Entonces para todo par de vértices w, z de $G_1 \cup G_2$ existe un camino de w a v y un camino de v a z (de longitud cero si alguno es v).



De la union de esos dos caminos se puede extraer un camino simple de w a z .

Conexión

Observaciones

1. Todo vértice de un grafo pertenece a alguna componente conexa.
2. Un grafo es conexo si y sólo si tiene una sola componente conexa.
3. Dos componentes conexas distintas de un grafo son disjuntas.

Demo de 3.: Supongamos que $v \in G_1 \cap G_2$. Entonces para todo par de vértices w, z de $G_1 \cup G_2$ existe un camino de w a v y un camino de v a z (de longitud cero si alguno es v).



De la unión de esos dos caminos se puede extraer un camino simple de w a z . Por lo tanto $G_1 \cup G_2$ es un subgrafo conexo, pero como G_1 y G_2 eran maximales, resulta $G_1 = G_2 = G_1 \cup G_2$. □

Distancia

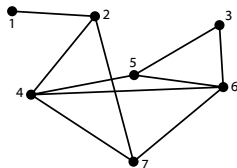
- La longitud de un camino se mide por la cantidad de aristas que lo componen.



Distancia

- La longitud de un camino se mide por la cantidad de aristas que lo componen.
- La distancia entre dos vértices v y w en G es la longitud del camino más corto entre v y w y se nota $d_G(v, w)$. Si el contexto no es ambiguo, se abrevia $d(v, w)$.

¿Cuál es la distancia entre 1 y 5?



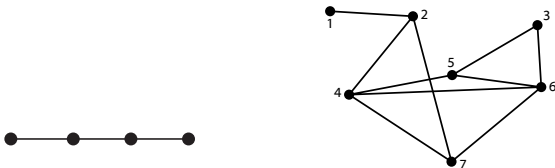
Distancia

- La **longitud de un camino** se mide por la cantidad de aristas que lo componen.
- La **distancia** entre dos vértices v y w en G es la longitud del camino más corto entre v y w y se nota $d_G(v, w)$. Si el contexto no es ambiguo, se abrevia $d(v, w)$.

¿Cuál es la distancia entre 1 y 5?

- El disco $D_k(v)$ de centro v y radio k ($k \geq 0$) es el conjunto de vértices de G que están a distancia menor o igual que k de v .

¿Cuáles son los discos con centro 1 en este grafo?



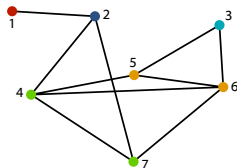
Distancia

- La **longitud de un camino** se mide por la cantidad de aristas que lo componen.
- La **distancia** entre dos vértices v y w en G es la longitud del camino más corto entre v y w y se nota $d_G(v, w)$. Si el contexto no es ambiguo, se abrevia $d(v, w)$.

¿Cuál es la distancia entre 1 y 5?

- El disco $D_k(v)$ de centro v y radio k ($k \geq 0$) es el conjunto de vértices de G que están a distancia menor o igual que k de v .

¿Cuáles son los discos con centro 1 en este grafo?



Distancia

La función de distancia cumple las siguientes propiedades para todo u, v, w pertenecientes a V :

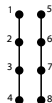
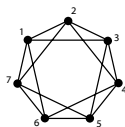
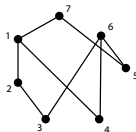
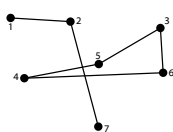
- $d(u, v) \geq 0$ y $d(u, v) = 0$ si y sólo si $u = v$.
- $d(u, v) = d(v, u)$.
- $d(u, w) \leq d(u, v) + d(v, w)$.

Grafos bipartitos

- Un grafo G es **bipartito** si $V(G) = V_1 \cup V_2$, con V_1 y V_2 disjuntos, y toda arista tiene un extremo en V_1 y otro en V_2 .

Grafos bipartitos

- Un grafo G es **bipartito** si $V(G) = V_1 \cup V_2$, con V_1 y V_2 disjuntos, y toda arista tiene un extremo en V_1 y otro en V_2 .
- ¿Cuáles de los siguientes grafos son bipartitos?



Teorema

Un grafo G es bipartito $\Leftrightarrow$ todos sus circuitos son pares.

Teorema

Un grafo G es bipartito $\Leftrightarrow$ todos sus circuitos son pares.

Demo:

$\Rightarrow$) Sabemos que $V(G) = V_1 \cup V_2$ y toda arista va de V_1 a V_2 . Sea $v_1, v_2, \dots, v_n$ un circuito en G . Si $v_1 \in V_1$ entonces los vértices de subíndice par tienen que pertenecer a V_2 y los de subíndice impar a V_1 . Como v_n es adyacente a v_1 , n tiene que ser par.

Teorema

Un grafo G es bipartito $\Leftrightarrow$ todos sus circuitos son pares.

Demo:

$\Rightarrow$) Sabemos que $V(G) = V_1 \cup V_2$ y toda arista va de V_1 a V_2 . Sea $v_1, v_2, \dots, v_n$ un circuito en G . Si $v_1 \in V_1$ entonces los vértices de subíndice par tienen que pertenecer a V_2 y los de subíndice impar a V_1 . Como v_n es adyacente a v_1 , n tiene que ser par.

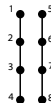
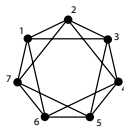
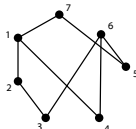
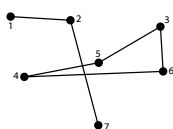
$\Leftarrow$) Sea v en $V(G)$. Definimos V_1 y V_2 como los vértices que están a distancia impar o par de v , respectivamente. Supongamos que no es una bipartición, o sea, existen z y w que están a ambos a distancia par o impar de v y son adyacentes. Como la diferencia entre las distancias es a lo sumo 1, entonces están a la misma distancia. Sea v' el primer vértice en común entre los caminos mínimos de w a v y de z a v . La longitud de los sub-caminos de w a v' y de z a v' tiene que ser la misma. Entonces esos sub-caminos y la arista wz forman un ciclo impar. □

Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .

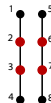
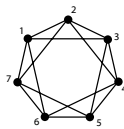
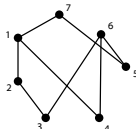
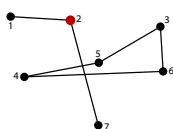
Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .
- ¿Qué vértices son puntos de corte en estos grafos?



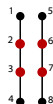
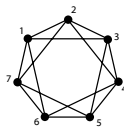
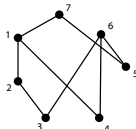
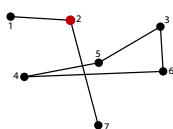
Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .
- ¿Qué vértices son puntos de corte en estos grafos?



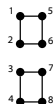
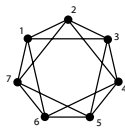
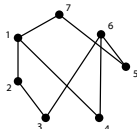
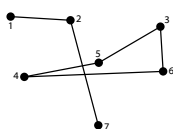
Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .
- ¿Qué vértices son puntos de corte en estos grafos?
- Un grafo es **biconexo** si es conexo y sin puntos de corte.



Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .
- ¿Qué vértices son puntos de corte en estos grafos?
- Un grafo es **biconexo** si es conexo y sin puntos de corte.
- ¿Cuáles de estos grafos son biconexos?

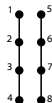
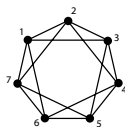
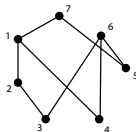
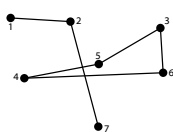


Conexión

- Un **bloque** o **componente biconexa** de un grafo es un subgrafo biconexo maximal.

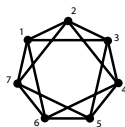
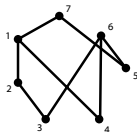
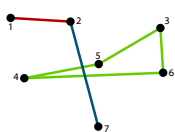
Conexión

- Un **bloque** o **componente biconexa** de un grafo es un subgrafo biconexo maximal.
- ¿Cuáles son los bloques en estos grafos?



Conexión

- Un **bloque** o **componente biconexa** de un grafo es un subgrafo biconexo maximal.
- ¿Cuáles son los bloques en estos grafos?



Conexión

Observaciones

1. Un grafo es biconexo si y sólo si tiene un solo bloque.
2. Dos bloques de un grafo comparten a lo sumo un vértice. En particular, cada arista pertenece a un único bloque.

Teorema

Sea G conexo y sea v un vértice de G . Son equivalentes:

1. El vértice v es un punto de corte de G .
2. Existen vértices u y w distintos de v tales que v está en todo camino entre u y w .
3. Existe una partición de $V - v$ en U y W tal que para todo u en U y para todo w en W , el punto v está en todo camino entre u y w .

Teorema

Sea G conexo y sea v un vértice de G . Son equivalentes:

1. El vértice v es un punto de corte de G .
2. Existen vértices u y w distintos de v tales que v está en todo camino entre u y w .
3. Existe una partición de $V - v$ en U y W tal que para todo u en U y para todo w en W , el punto v está en todo camino entre u y w .

Demo: $1 \Rightarrow 3$) Si v es punto de corte $\Rightarrow G - v$ es desconexo. Sea U una componente conexa de $G - v$ y W los vértices restantes. Sean $u \in U$ y $w \in W$; como están en componentes conexas distintas de $G - v$, todo camino en G entre ellos contiene a v .

Teorema

Sea G conexo y sea v un vértice de G . Son equivalentes:

1. El vértice v es un punto de corte de G .
2. Existen vértices u y w distintos de v tales que v está en todo camino entre u y w .
3. Existe una partición de $V - v$ en U y W tal que para todo u en U y para todo w en W , el punto v está en todo camino entre u y w .

Demo: $1 \Rightarrow 3$) Si v es punto de corte $\Rightarrow G - v$ es desconexo. Sea U una componente conexa de $G - v$ y W los vértices restantes. Sean $u \in U$ y $w \in W$; como están en componentes conexas distintas de $G - v$, todo camino en G entre ellos contiene a v .

$3 \Rightarrow 2$) Tomamos u en U y w en W .

Teorema

Sea G conexo y sea v un vértice de G . Son equivalentes:

1. El vértice v es un punto de corte de G .
2. Existen vértices u y w distintos de v tales que v está en todo camino entre u y w .
3. Existe una partición de $V - v$ en U y W tal que para todo u en U y para todo w en W , el punto v está en todo camino entre u y w .

Demo: $1 \Rightarrow 3$) Si v es punto de corte $\Rightarrow G - v$ es desconexo. Sea U una componente conexa de $G - v$ y W los vértices restantes. Sean $u \in U$ y $w \in W$; como están en componentes conexas distintas de $G - v$, todo camino en G entre ellos contiene a v .

$3 \Rightarrow 2$) Tomamos u en U y w en W .

$2 \Rightarrow 1$) Si v está en todo camino de u a w , entonces no existe un camino entre u y w en $G - v$. Por lo tanto $G - v$ no es conexo, y v es punto de corte de G . □

Conexión

- Un **punte** de un grafo G es una arista e tal que $G - e$ tiene más componentes conexas que G .

Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .
- Sea G conexo, v un punto de corte y e un puente. ¿Puede ser que $G - v$ tenga más de dos componentes conexas? ¿Y $G - e$?

Conexión

- Un **punto de corte** de un grafo G es una arista e tal que $G - e$ tiene más componentes conexas que G .
- Sea G conexo, v un punto de corte y e un puente. ¿Puede ser que $G - v$ tenga más de dos componentes conexas? ¿Y $G - e$?
- ¿Existe algún grafo biconexo que tenga un puente?

Conexión

- Un **punto de corte** de un grafo G es un vértice v tal que $G - v$ tiene más componentes conexas que G .
- Sea G conexo, v un punto de corte y e un puente. ¿Puede ser que $G - v$ tenga más de dos componentes conexas? ¿Y $G - e$?
- ¿Existe algún grafo biconexo que tenga un puente?

Rta: Sólo el grafo formado por una única arista. Si $e = vw$ es un puente en G , entonces las componentes conexas de $G - e$ son G_1 y G_2 , donde $v \in G_1$ y $w \in G_2$. Notemos que v es punto de corte en G salvo que $G_1 = \{v\}$ y w es punto de corte en G salvo que $G_2 = \{w\}$. Entonces, si G es biconexo, $V(G) = \{v, w\}$ y $E(G) = \{e\}$. □

Teorema

Sea G conexo y sea $e = ij$ una arista de G . Son equivalentes:

1. La arista e es un puente de G .
2. La arista e no está en ningún ciclo de G .
3. Existen vértices u y v tales que e está en todo camino entre u y v .

Teorema

Sea G conexo y sea $e = ij$ una arista de G . Son equivalentes:

1. La arista e es un puente de G .
2. La arista e no está en ningún ciclo de G .
3. Existen vértices u y v tales que e está en todo camino entre u y v .

Demo: $1 \Rightarrow 2$) Si e está en un ciclo C , entonces $C - e$ es un camino P entre i y j . En cualquier camino entre dos vértices u y v , la arista e podría ser reemplazada por el camino P . Luego e no es puente.

Teorema

Sea G conexo y sea $e = ij$ una arista de G . Son equivalentes:

1. La arista e es un puente de G .
2. La arista e no está en ningún ciclo de G .
3. Existen vértices u y v tales que e está en todo camino entre u y v .

Demo: $1 \Rightarrow 2$) Si e está en un ciclo C , entonces $C - e$ es un camino P entre i y j . En cualquier camino entre dos vértices u y v , la arista e podría ser reemplazada por el camino P . Luego e no es puente.

$2 \Rightarrow 3$) Sean i y j los extremos de e . Si para todo par de vértices u, v existe un camino que los une y no pasa por e , en particular existe un camino P entre i y j que no usa e . Pero entonces $P \cup e$ es un ciclo.

Teorema

Sea G conexo y sea $e = ij$ una arista de G . Son equivalentes:

1. La arista e es un puente de G .
2. La arista e no está en ningún ciclo de G .
3. Existen vértices u y v tales que e está en todo camino entre u y v .

Demo: $1 \Rightarrow 2$) Si e está en un ciclo C , entonces $C - e$ es un camino P entre i y j . En cualquier camino entre dos vértices u y v , la arista e podría ser reemplazada por el camino P . Luego e no es puente.

$2 \Rightarrow 3$) Sean i y j los extremos de e . Si para todo par de vértices u, v existe un camino que los une y no pasa por e , en particular existe un camino P entre i y j que no usa e . Pero entonces $P \cup e$ es un ciclo.

$3 \Rightarrow 1$) Si e está en todo camino de u a v , entonces no existe un camino entre u y v en $G - e$. Por lo tanto $G - e$ no es conexo, y e es un puente de G . □

Conexión en digrafos

- Un **camino orientado** en un grafo orientado es una sucesión de arcos $e_1 e_2 \dots e_k$ tal que el primer elemento del par e_i coincide con el segundo de e_{i-1} y el segundo elemento de e_i con el primero de e_{i+1} $i = 2, \dots, k - 1$.
- Un **cicuito orientado** en un grafo orientado es un camino orientado que comienza y termina en el mismo vértice.
- Un digrafo se dice **fuertemente conexo** si para todo par de vértices u, v existe un camino orientado de u a v y otro de v a u .

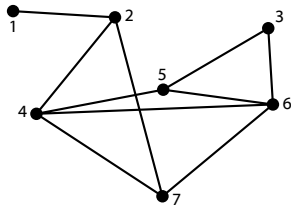
Representación de grafos

- Matriz de adyacencia
- Matriz de incidencia
- Listas de vecinos (para cada vértice, se indica la lista de sus vecinos).
- Cantidad de vértices y lista de aristas (se asumen los vértices numerados de 1 a n).

Matriz de adyacencia

Dado un grafo G cuyos vértices están numerados de 1 a n , definimos la matriz de adyacencia de G como $M \in \{0, 1\}^{n \times n}$ donde $M(i, j) = 1$ si los vértices i y j son adyacentes y 0 en otro caso.

Ej:



$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

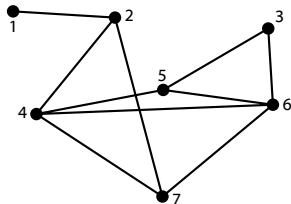
Propiedades de la matriz de adyacencia

- Si A_G es la matriz de adyacencia de G , entonces $A_G^k[i, j]$ es la cantidad de caminos (no necesariamente simples) distintos de longitud k entre i y j .
- En particular, $A_G^2[i, i] = d_G(i)$.
- Y además, un grafo G es bipartito si y sólo si $A_G^k[i, i] = 0$ para todo k impar, o sea, si la diagonal de la matriz $\sum_{j=0}^{n/2} A_G^{2j+1}$ tiene la diagonal nula.
- Un grafo G es conexo si y sólo si la matriz $\sum_{j=1}^n A_G^j$ no tiene ceros.

Matriz de incidencia

Numerando las aristas de G de 1 a m , definimos la matriz de incidencia de G como $M \in \{0,1\}^{m \times n}$ donde $M(i,j) = 1$ si el vértice j es uno de los extremos de la arista i y 0 en otro caso.

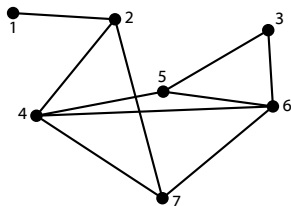
Ej:



$$\begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

Listas de vecinos

Ej:



$$L_1 : 2$$

$$L_2 : 1 \rightarrow 4 \rightarrow 7$$

$$L_3 : 5 \rightarrow 6$$

$$L_4 : 2 \rightarrow 5 \rightarrow 6 \rightarrow 7$$

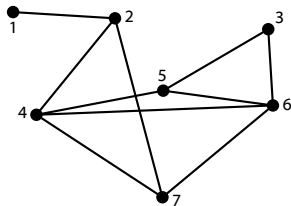
$$L_5 : 3 \rightarrow 4 \rightarrow 6$$

$$L_6 : 3 \rightarrow 4 \rightarrow 5 \rightarrow 7$$

$$L_7 : 2 \rightarrow 4 \rightarrow 6$$

Lista de aristas

Ej:



Cant. vértices: 7

Aristas: $(1,2), (2,4), (2,7), (3,5),$
 $(3,6), (4,5), (4,6), (4,7), (5,6), (6,7)$

Es una estructura útil para el algoritmo de Kruskal, por ejemplo. También es la que se usa en general para almacenar un grafo en un archivo de texto.

Algoritmos para recorrido de grafos

Descripción del problema a grandes rasgos:

Tengo un grafo descrito de cierta forma y quiero recorrer sus vértices para:

- encontrar un vértice que cumpla determinada propiedad.
- calcular una propiedad de los vértices, por ejemplo la distancia a un vértice dado.
- calcular una propiedad del grafo, por ejemplo sus componentes conexas.
- obtener un orden de los vértices que cumpla determinada propiedad.

Estructuras de datos

Una **pila** (*stack*) es una estructura de datos donde el último en entrar es el primero en salir. Las tres operaciones básicas de una pila son:

- **apilar** (*push*), que coloca un objeto en la parte superior de la pila,
- **tope** (*top*) que permite ver el objeto superior de la pila y
- **desapilar** (*pop*), que elimina de la pila el tope.



Estructuras de datos

Una **cola** (*queue*) es una estructura de datos donde el primero en llegar es el primero en salir.

Las tres operaciones básicas de una cola son:

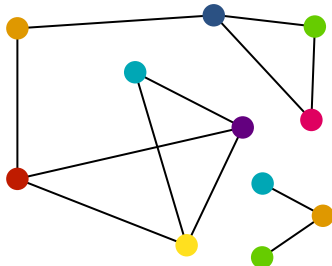
- **encolar** (*push*), que coloca un objeto al final de la cola,
- **primero** (*first*) que permite ver el primer objeto de la cola y
- **desencolar** (*pop*), que elimina de la cola el primer objeto.



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

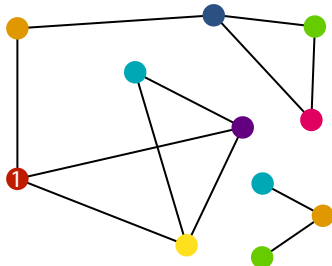
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

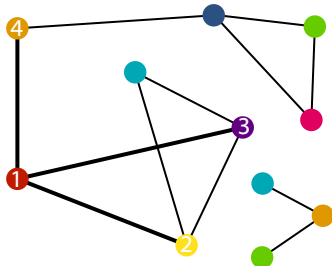
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

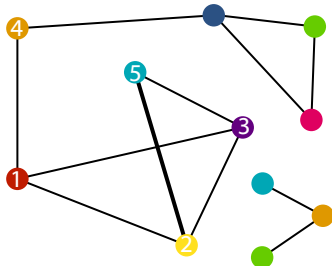
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

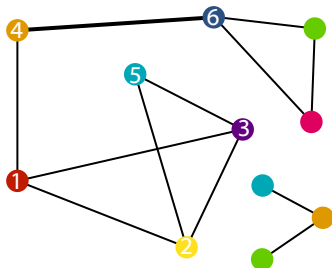
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

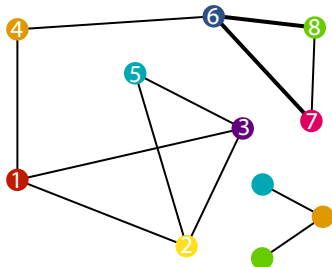
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

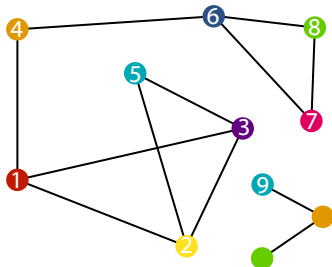
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

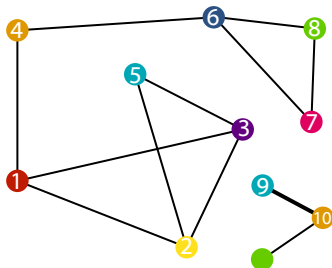
Ejemplo:



Recorrido BFS

BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

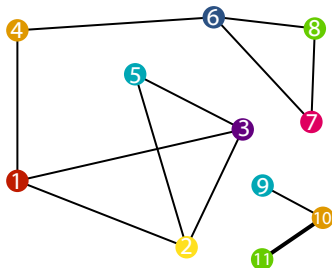
Ejemplo:



Recorrido BFS

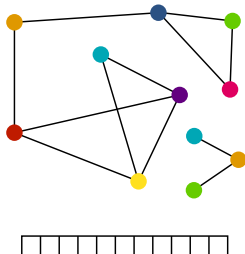
BFS: *Breadth-First Search*, o sea, recorrido a lo ancho. A partir de un vértice, recorre sus vecinos, luego los vecinos de sus vecinos, y así sucesivamente.... si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:



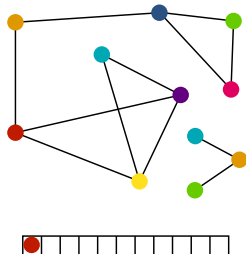
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



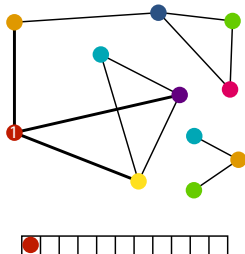
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



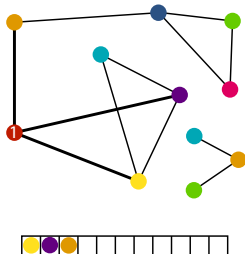
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



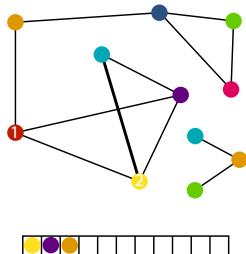
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



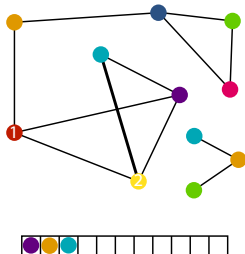
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



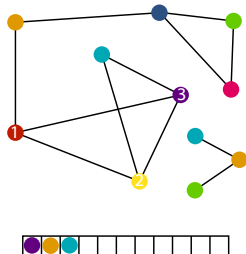
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



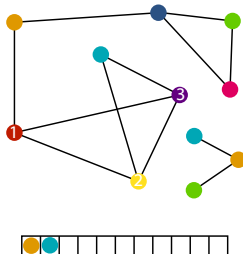
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



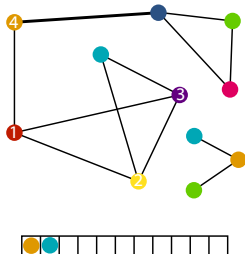
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    C.vaciar();
    C.encolar(v);
    Mientras C no sea vacía
        w = C.primer();
        ord++;
        orden(w)=ord;
        C.desencolar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                C.encolar(z);
```



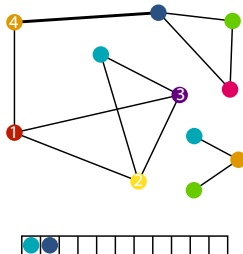
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



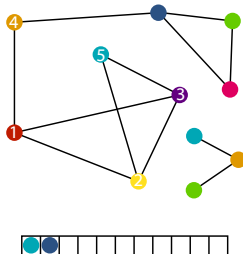
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



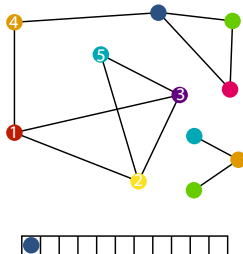
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



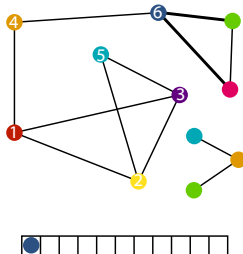
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    C.vaciar();
    C.encolar(v);
    Mientras C no sea vacía
        w = C.primer();
        ord++;
        orden(w)=ord;
        C.desencolar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                C.encolar(z);
```



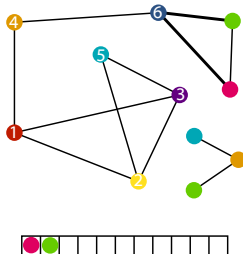
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



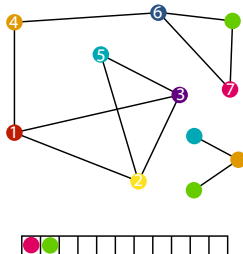
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



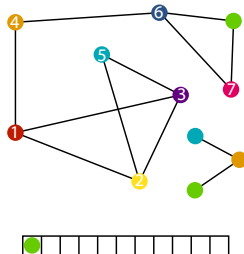
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



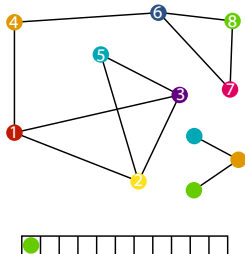
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



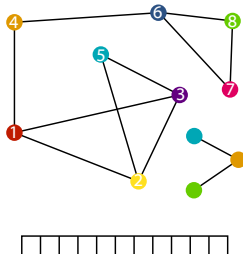
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



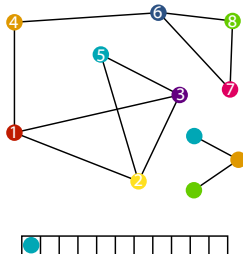
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



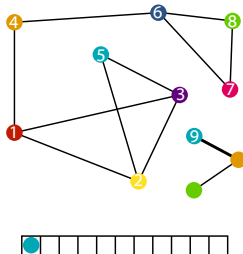
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



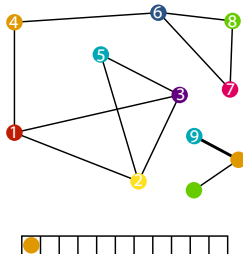
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



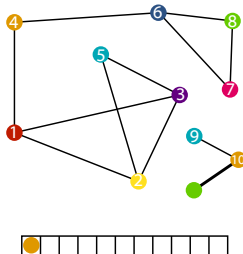
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



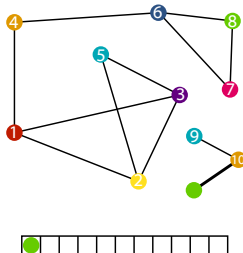
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



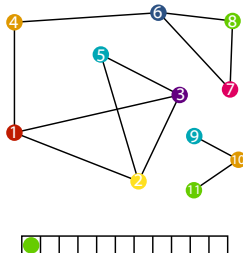
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



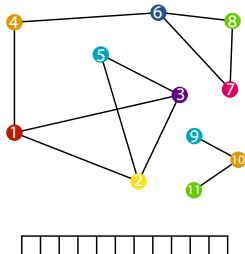
Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



Recorrido BFS: pseudocódigo

```
cola C; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  C.vaciar();  
  C.encolar(v);  
  Mientras C no sea vacía  
    w = C.primer();  
    ord++;  
    orden(w)=ord;  
    C.desencolar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        C.encolar(z);
```



Recorrido BFS: propiedades

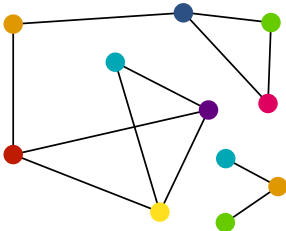
Notemos que cada vez que comenzamos desde un vértice nuevo, es porque los vértices que quedan sin visitar no son alcanzables desde los vértices visitados. Entonces lo que estamos encontrando son las diferentes **componentes conexas** del grafo.

Si ponemos un contador que se incremente luego de la operación *elegir un vértice no visitado*, lo que obtenemos es la cantidad de componentes conexas del grafo (y con algunas leves modificaciones podemos obtener las componentes en sí).

Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** .

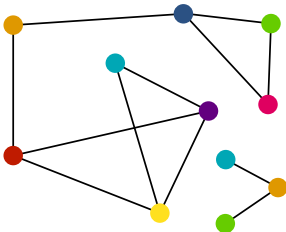
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

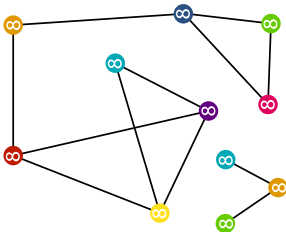
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

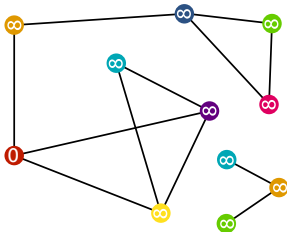
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

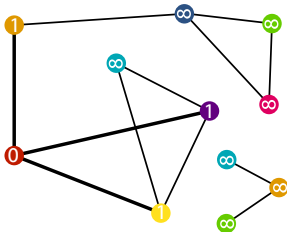
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

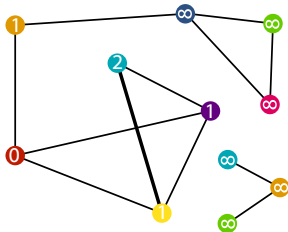
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

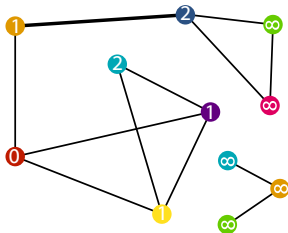
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

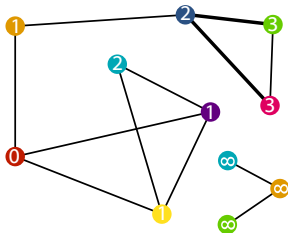
Ejemplo:



Recorrido BFS: propiedades

Si partimos desde un vértice v , modificando levemente el código podemos calcular **para cada vértice su distancia a v** . Inicialmente todos los vértices tienen rótulo ∞ . Rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún tienen rótulo ∞ , los rotulamos con el rótulo de w más 1. (Notemos que los que no sean alcanzados desde v , o sea, no pertenecen a la componente conexa de v , tienen distancia infinita a v .)

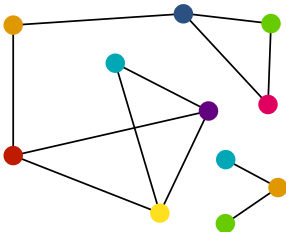
Ejemplo:



Recorrido BFS: reconocimiento de grafos bipartitos

Utilizamos una idea parecida a la del algoritmo anterior en cada componente conexas: rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún no tienen rótulo, los rotulamos con 1 menos el rótulo de w . De esta manera queda determinada la partición, de ser bipartito el grafo. Queda entonces verificar que ningún par de vértices con el mismo rótulo sean adyacentes (ese chequeo se puede ir haciendo en cada paso).

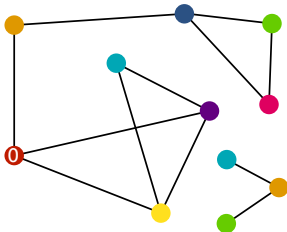
Ejemplo:



Recorrido BFS: reconocimiento de grafos bipartitos

Utilizamos una idea parecida a la del algoritmo anterior en cada componente conexas: rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún no tienen rótulo, los rotulamos con 1 menos el rótulo de w . De esta manera queda determinada la partición, de ser bipartito el grafo. Queda entonces verificar que ningún par de vértices con el mismo rótulo sean adyacentes (ese chequeo se puede ir haciendo en cada paso).

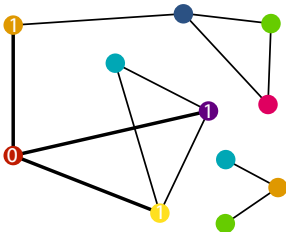
Ejemplo:



Recorrido BFS: reconocimiento de grafos bipartitos

Utilizamos una idea parecida a la del algoritmo anterior en cada componente conexas: rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún no tienen rótulo, los rotulamos con 1 menos el rótulo de w . De esta manera queda determinada la partición, de ser bipartito el grafo. Queda entonces verificar que ningún par de vértices con el mismo rótulo sean adyacentes (ese chequeo se puede ir haciendo en cada paso).

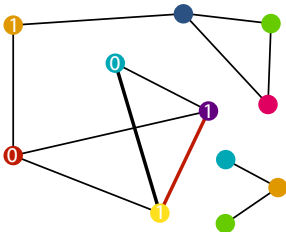
Ejemplo:



Recorrido BFS: reconocimiento de grafos bipartitos

Utilizamos una idea parecida a la del algoritmo anterior en cada componente conexa: rotulamos el vértice v con 0 y luego, al visitar los vecinos de cada vértice w que aún no tienen rótulo, los rotulamos con 1 menos el rótulo de w . De esta manera queda determinada la partición, de ser bipartito el grafo. Queda entonces verificar que ningún par de vértices con el mismo rótulo sean adyacentes (ese chequeo se puede ir haciendo en cada paso).

Ejemplo:

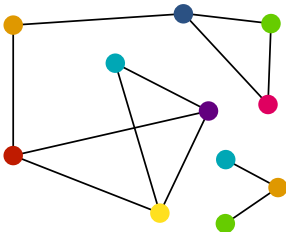


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

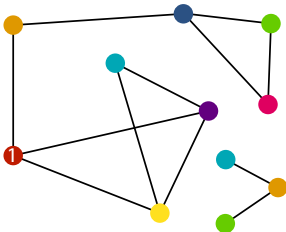


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

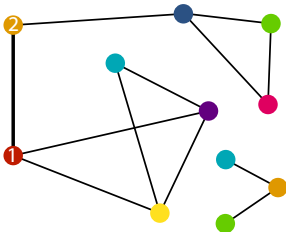


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

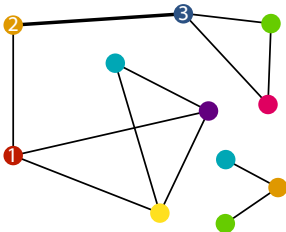


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

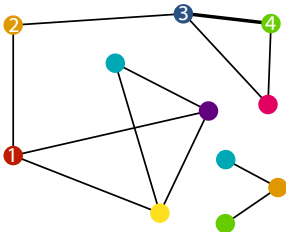


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

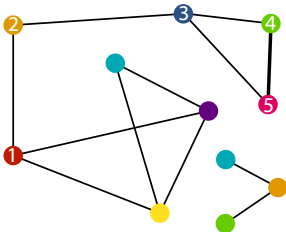


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

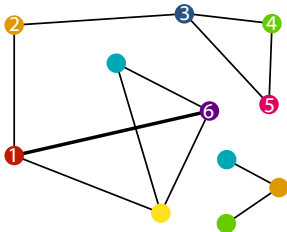


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

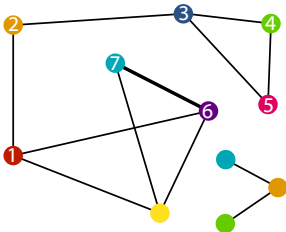


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

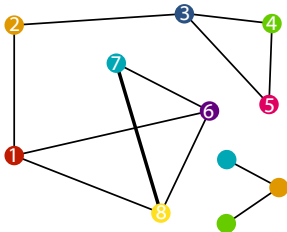


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

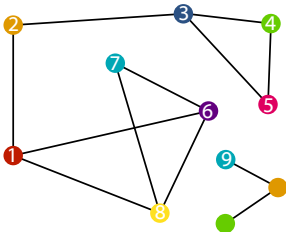


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

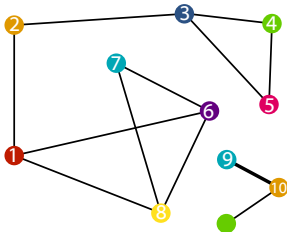


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:

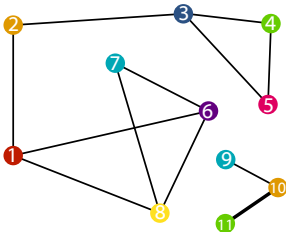


Recorrido DFS

DFS: *Depth-First Search*, o sea, recorrido en profundidad. A partir de un vértice, comienza por alguno de sus vecinos, luego un vecino de éste, y así sucesivamente hasta llegar a un vértice que ya no tiene vecinos sin visitar. Sigue entonces por algún vecino del último vértice visitado que aún tiene vecinos sin visitar.

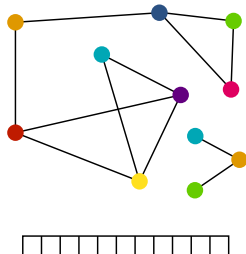
Igual que en BFS, si al finalizar quedan vértices sin visitar, se repite a partir de un vértice no visitado.

Ejemplo:



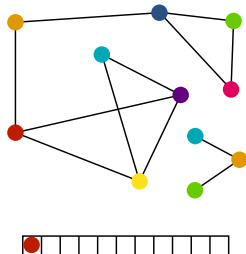
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



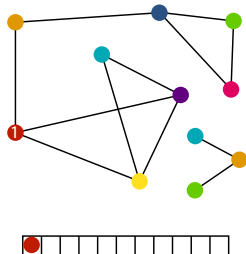
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



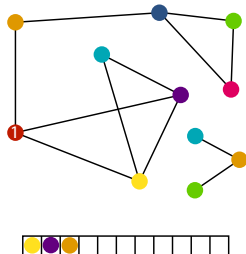
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



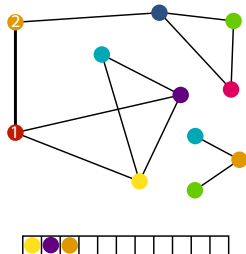
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

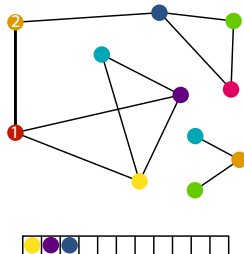
```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

```

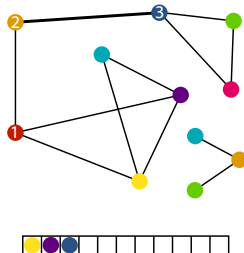
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

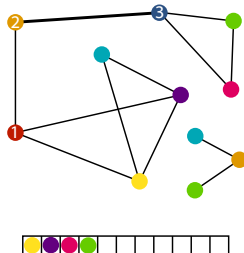
```

pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

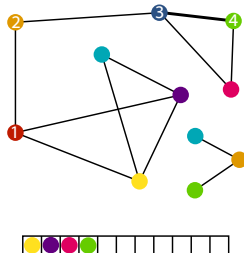
```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

```

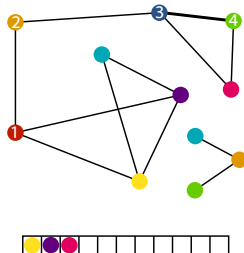
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

```

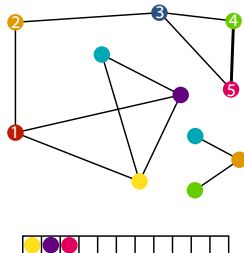
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

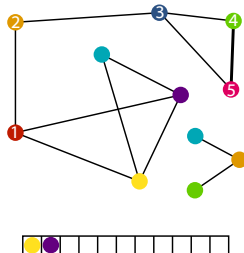
```

pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



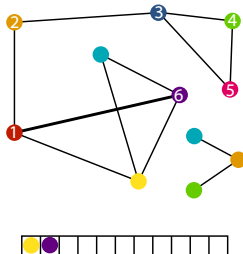
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



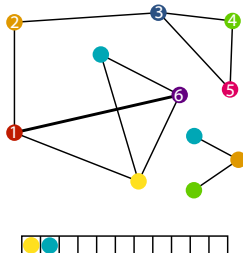
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



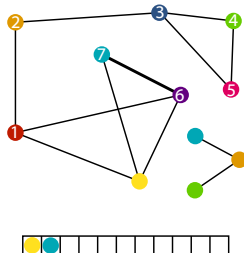
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

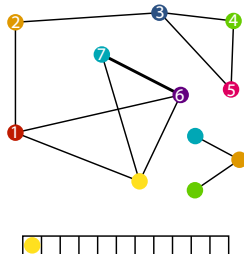
```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

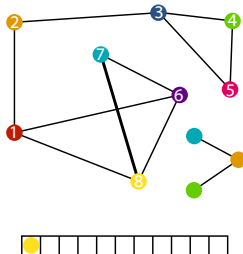
```

pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



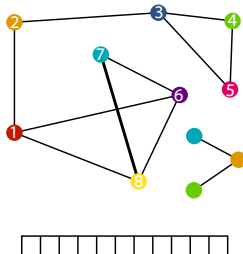
Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

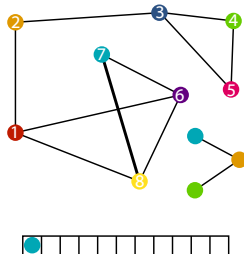
```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Recorrido DFS: pseudocódigo

```

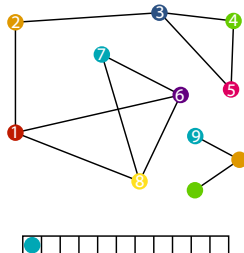
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

```

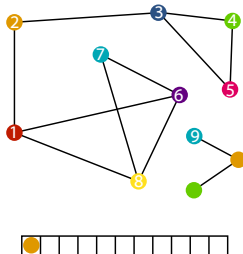
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

```

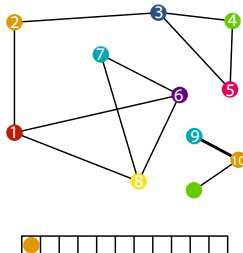
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

```

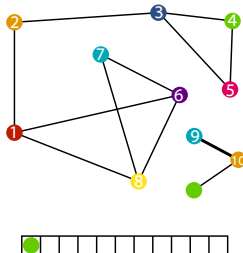
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

```

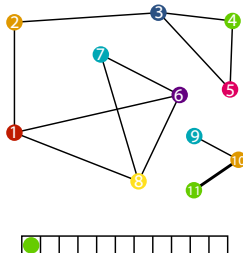
pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

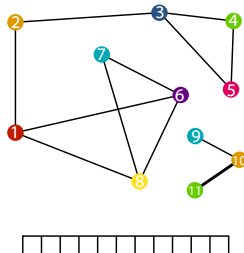
```

pila P; vértices v, w; ord = 0;
Mientras queden vértices sin visitar
    v = elegir un vértice no visitado;
    marcar v;
    P.vaciar();
    P.apilar(v);
    Mientras P no sea vacía
        w = P.tope();
        ord++;
        orden(w)=ord;
        P.desapilar();
        Para cada vecino z de w
            Si z no está visitado
                marcar z;
                P.apilar(z);
    
```



Recorrido DFS: pseudocódigo

```
pila P; vértices v, w; ord = 0;  
Mientras queden vértices sin visitar  
  v = elegir un vértice no visitado;  
  marcar v;  
  P.vaciar();  
  P.apilar(v);  
  Mientras P no sea vacía  
    w = P.tope();  
    ord++;  
    orden(w)=ord;  
    P.desapilar();  
    Para cada vecino z de w  
      Si z no está visitado  
        marcar z;  
        P.apilar(z);
```



Propiedades

- Al igual que BFS, este algoritmo puede modificarse para calcular las componentes conexas de un grafo y/o su cantidad.

Propiedades

- Al igual que BFS, este algoritmo puede modificarse para calcular las componentes conexas de un grafo y/o su cantidad.
- El grafo en si puede no estar totalmente almacenado, pero tener una descripción implícita que permita generar bajo demanda los vecinos de un vértice. Un ejemplo típico de este caso es cuando el grafo es el árbol de posibilidades de un problema de búsqueda exhaustiva. El recorrido DFS es el que se utiliza habitualmente para hacer *backtracking*, ya que permite encontrar algunas soluciones rápido y usarlas para “podar” (es decir, no recorrer) ciertas ramas del árbol.

Propiedades

- Al igual que BFS, este algoritmo puede modificarse para calcular las componentes conexas de un grafo y/o su cantidad.
- El grafo en si puede no estar totalmente almacenado, pero tener una descripción implícita que permita generar bajo demanda los vecinos de un vértice. Un ejemplo típico de este caso es cuando el grafo es el árbol de posibilidades de un problema de búsqueda exhaustiva. El recorrido DFS es el que se utiliza habitualmente para hacer *backtracking*, ya que permite encontrar algunas soluciones rápido y usarlas para “podar” (es decir, no recorrer) ciertas ramas del árbol.
- BFS puede ser útil en ese contexto para encontrar el camino más corto hacia la salida de un laberinto, por ejemplo, aún si el laberinto viene descrito de manera implícita.

Estructuras de datos y complejidad

- Para estos algoritmos conviene en general tener el grafo dado por listas de adyacencia.

Estructuras de datos y complejidad

- Para estos algoritmos conviene en general tener el grafo dado por listas de adyacencia.
- De esta manera, el *Para cada vecino z de w* puede ser realizado en orden $O(d(w))$.

Estructuras de datos y complejidad

- Para estos algoritmos conviene en general tener el grafo dado por listas de adyacencia.
- De esta manera, el *Para cada vecino z de w* puede ser realizado en orden $O(d(w))$.
- Con lo cual, si el marcar, asignar orden y las operaciones de cola y pila se realizan en orden constante, la complejidad total será de $O(n + m)$, para un grafo de n vértices y m aristas. El n viene del recorrido secuencial que va identificando si quedan vértices no marcados.

Estructuras de datos y complejidad

- Para estos algoritmos conviene en general tener el grafo dado por listas de adyacencia.
- De esta manera, el *Para cada vecino z de w* puede ser realizado en orden $O(d(w))$.
- Con lo cual, si el marcar, asignar orden y las operaciones de cola y pila se realizan en orden constante, la complejidad total será de $O(n + m)$, para un grafo de n vértices y m aristas. El n viene del recorrido secuencial que va identificando si quedan vértices no marcados.
- Si en cambio el grafo está dado por matriz de adyacencia, la complejidad será $O(n^2)$, porque buscar los vecinos cuesta $O(n)$.